handnet: Neural Networks with Hand-Written Weights

Birger Moëll
Uppsala University, KTH, AI Sweden

Draft, November 2026

Abstract

We study neural networks whose weights are *designed* rather than learned. The architecture is standard (embeddings, convolutions, ReLU recurrences, multilayer perceptrons), the tensors are allocated at a fixed parameter budget, and every non-zero weight is placed by reasoning about the task; the vast majority of the budget stays zero. We describe a small toolbox of constructions (pattern kernels, first-of-run counting, run-length counters as a ReLU recurrence with a closed form, capped detectors from ReLU pairs, min gates, count-then-threshold) that lets ordinary task knowledge be written directly as weights, and we report experiments on a multilingual text-quality network of 100,331 parameters (627 non-zero) that reaches AUC 0.86–0.95 at separating kept from removed FineWeb-2 documents across eight scripts without having seen a document. We discuss where the approach stops (knowledge that cannot be articulated in the input features, such as morphology), and how designed networks relate to trained ones as initialisations, baselines and specifications.

1 Introduction

2 Related work

3 Method

3.1 Setting

A hand-written network is an ordinary feed-forward or recurrent network f_θ whose parameter tensors are allocated at a fixed budget and initialised to zero, after which individual entries are set by the author. We require that (i) every layer is a standard operation (affine map, ReLU, convolution, ReLU recurrence, mean/max pooling), so that θ loads into any deep-learning framework unchanged, and (ii) every non-zero entry carries a comment stating why it has its value. We report two numbers for each network: the allocated parameter count $|\theta|$ and the number of non-zero entries $\|\theta\|_0$.

3.2 Input representation

The weights are only as readable as the features they read. We therefore use inputs whose meaning is fixed a priori and make the first embedding the identity. For text, each character is mapped to a binary attribute vector (Unicode general category, case, script block, punctuation class, combining mark, a same-as-previous-character bit, and a few task-specific markers such as the lead and tail bytes of UTF-8 mojibake). No vocabulary is involved, so one set of weights applies to every language and script.

3.3 Constructions

Six constructions account for every non-zero weight in the networks reported here.

Pattern kernels. A convolution channel with weight 1 on each required (offset, attribute) pair and bias $-(n-1)$ fires under ReLU only when all n attributes are present: $\mathrm{relu}(\mathrm{terminal}_{t-2} + \mathrm{space}_{t-1} + \mathrm{upper}_t - 2)$ detects a sentence start in cased scripts. A large negative weight on an attribute implements exclusion.

First-of-run counting. $\mathrm{relu}(x_t - \mathrm{same_prev}_t)$ is 1 only at the first character of a run of x, so that a separator line of eighty dashes contributes one count rather than eighty.

Run-length counters. A ReLU recurrence $h_t = \mathrm{relu}(h_{t-1} + u_t)$ with unit self-weight and input $u_t = \mathrm{inc}_t - B \cdot \mathrm{reset}_t$ (B large) counts increments since the last reset: characters since the last space (word length), since the last sentence end (sentence length), words since the last sentence end. Because the self-weight is exactly 1, the recurrence has the closed form $h_t = S_t - \min(0, \min_{k \le t} S_k)$ with S the cumulative sum of u (the Lindley recursion), so inference is vectorised; the step-by-step recurrence is used for the PyTorch export and the two are tested to agree.

Capped detectors. A penalty that rises from 0 at threshold a to its full value at b and saturates is $\min(\mathrm{relu}(x), 1) = \mathrm{relu}(x) - \mathrm{relu}(x-1)$ with x an affine function of the pooled features scaled so that $x = 1$ is "fully bad". The next layer multiplies the pair by a point budget. Every penalty and reward term is one such unit, and the sum of the named terms is the output; this is what makes `explain()` exact rather than approximate.

Gates. A detector that is meaningless for some input mode (word length when the script has no spaces; capitalisation when the script has no case) receives a weight of -100 on the rate of that mode. A soft gate $\min(g, p)$, used to switch rate-based penalties on only once the document is long enough for rates to be meaningful, costs one extra layer: $p - \mathrm{relu}(p - g)$.

Count-then-threshold. "More than one script present" is one capped unit per script, summed with bias -1 and passed through ReLU; the output is the number of extra scripts, itself capped.

3.4 Budgets and naming

The output is a base value plus named reward terms minus named penalty terms, grouped into sub-scores and clipped to the target range by a final $\mathrm{relu}(z) - \mathrm{relu}(z - 100)$. Budgets are chosen in output units: in the text-quality network a markup rate above 10% can cost at most 20 of 100 points, mojibake at most 35, and the rewards for the shape of running prose sum to 28, so that clean prose reaches 100 and a table reaches 0.

3.5 Development protocol

We develop against three sources in order: (1) hand-written samples across scripts and junk types, each a test case with a stated reason; (2) a labelled external set the network has never seen; (3) the worst mistakes in both directions on (2), fixed at the level of the mechanism rather than the sample. Every fix is recorded as a comment in the code and a line in Section 4.2. The network never sees a training signal; the author does.

3.6 Protocol for comparing designed and trained weights

The sentiment and MNIST experiments follow a protocol that turns "the designed network works" into "the designed network is worth N labels": (1) freeze a differentiable architecture, building min and clipping from ReLUs so that a trained model of the same shape is a meaningful baseline; (2) name every unit; (3) write all non-zero weights *blind*, before any labelled example from the benchmark is seen (general domain knowledge is allowed, the data is not); (4) evaluate once on the sealed test set and print only aggregates; (5) allow a bounded number of *revisions*, each of which inspects a development sample, states a hypothesis about language or shape, changes weights and is reported as its own row with the number of labelled examples it saw, the earlier file being kept so the row can be regenerated; (6) train the same architecture on 2^k labels for a range of k and plot. The author—here a language model, Claude Fable 5.1, in a chat session—may not call any optimiser, may not grid-search over labelled data, and may not import pre-trained resources. Fixed operations are allowed when parameter-free or hand-set (a Gaussian blur, ℓ_2 normalisation, max-pooling over offsets).

4 Experiments

4.1 A 100k-parameter multilingual text-quality network

Task. Score a document in $[0, 100]$ for usability as text in the pretraining-filter sense: clean running prose versus markup, tables, logs, mojibake, repetition, shouting, navigation lists, keyword lists, fragments and mixed scripts. The score must be defined for every language and script with one set of weights.

Architecture. Characters $\to$ 40 binary Unicode attributes $\to$ identity embedding (40×40) $\to$ width-5 convolution with 48 channels (18 pass-through rates, 17 pattern detectors, 13 script rates) $\to$ a 16-unit ReLU recurrence of run-length counters $\to$ pooling (channel means, counter maxima and means, log length, bias; 82 features) $\to$ an MLP $82 \to 512 \to 86 \to 16 \to 2 \to 1$. Layer 1 holds 70 capped detectors, 10 script-presence units and 2 length gates; layer 2 the named sub-scores, gates, reward and mixed-script count; layer 3 the $\min(\text{gate}, \text{penalty})$ terms; layer 4 the base-plus-terms score and its clip. In total $|\theta| = 100{,}331$ and $\|\theta\|_0 = 627$.

Script handling. Word-based detectors are gated off by the rate of Han, Kana and Thai-family characters; case-based detectors by Arabic, Indic, Hangul and Georgian (Unicode classes Mkhedruli as lowercase; it is written caseless). Sentence starts have a cased kernel (`.  A`) and a caseless kernel (`, . `). Combining marks count as letters so that Indic and Arabic vowel signs do not depress the letter rate. Han and Kana count as one script. A CJK character is weighted as roughly 2.5 Latin characters in the length terms.

Evaluation data. FineWeb-2 publishes, per language, the documents that survived its filtering pipeline and the documents its filters removed. We take the first 300 of each from the first shard for Swedish, German, Finnish, Turkish, Georgian, Arabic, Hindi and Japanese (Latin, Georgian, Arabic, Devanagari and Japanese scripts) and report AUC for kept > removed. The network was never run on these documents during development except through the error analysis in Section 4.2, which used the same sample; a held-out shard should be added before publication (TODO).

	swe	deu	fin	tur	kat	arb	hin	jpn
AUC	0.928	0.901	0.941	0.901	0.949	0.883	0.857	0.896
mean kept	95.2	95.6	97.4	94.5	95.2	88.6	91.9	90.8
mean removed	68.8	74.1	64.8	70.5	58.4	64.4	71.3	61.3

Table 1: FineWeb-2 kept vs removed, 300 + 300 documents per language. Scoring takes about 7 ms per document in numpy on one CPU core.

Hand-written samples. Encyclopaedic paragraphs in fourteen languages (Swedish, English, German, Georgian, Finnish, Japanese, Chinese, Arabic, Hindi, Russian, Greek, Thai, Korean, Turkish) score 85–97. HTML scores 23, UTF-8 mojibake 48, character repetition 35, shouting 37, a markdown table 0, a navigation list 36, a server log 9, a comma-separated keyword list 57, a list of URLs 47, U+FFFD soup 32, source code 17, a paragraph mixing five scripts 47, a 110-character string without spaces 41 and emoji spam 24.

Where it fails, by construction. On synthetic grammar pairs from the OELLM quality-gate data (a clean one-to-three-sentence snippet against the same snippet with one inflection change, adjacent-word swap, typo, duplicated word or deleted function word; Swedish, Norwegian, Georgian), the network is at chance, and most pairs are exact ties: the attribute representation carries no morphology or word order. Deleted function words are detected slightly above chance (62–86%) through length alone. The highest-scoring *removed* FineWeb-2 documents are fluent machine-translated hostel listings and site boilerplate, which have the surface shape of prose. Both failure modes are exactly the signal a trained judge provides, and the two are complementary: the designed network is $\sim$1000$\times$ cheaper than a 0.3B judge and can run first.

4.2 Development log

Each entry is a mistake found on external data and the mechanism that fixed it.

1. Hindi prose scored 61: Devanagari vowel signs are combining marks, so the letter rate was 45%. Fix: combining marks count as letters in the letter-rate terms (zalgo detection stays separate and is gated off for Indic, Arabic and Thai).

2. Japanese prose lost 12 points as "mixed scripts": Han and Kana were two scripts. Fix: one presence unit reads both rates.

3. Korean and Georgian earned no sentence reward: Korean puts a space after the full stop before a caseless letter; Georgian is Unicode-lowercase. Fix: a second sentence kernel, and Mkhedruli mapped to the caseless attribute.

4. An 80-character separator line cost 56 points through four detectors (repetition, punctuation runs, symbol streak, giant token). Fix: first-of-run counting for markup, symbol and dash rates; repetition and punctuation-run kernels exclude separator characters; the word-length counter increments only on letters and digits; the symbol-streak threshold raised.

5. One all-caps headline cost the full 15 shouting points. Fix: the caps-run rate needs about 15% of the text before the penalty saturates.

6. Short texts were hit by every rate-based penalty at once (a 17-character sentence-splitter fragment scored 3). Fix: two length gates as min(gate, penalty); composition penalties switch on between 30 and 100 characters, structure penalties between 100 and 300.

7. The replacement character U+FFFD was classed as a symbol, not garbage. Fix: attribute order.

8. Chinese and Japanese lost 4–7 points for "comma-heavy" text (ideographic commas are dense). Fix: the detector is gated off for spaceless scripts.

No entry changed the FineWeb-2 AUC by more than ±0.01; the fixes removed false positives on individual documents (e.g. a Georgian medical page from 66 to 96) without moving the aggregate.

4.3 Ablation by sub-score

Switching off one named sub-score at a time (zeroing its layer-2 unit; `examples/text_quality/ablate.py`) shows that on FineWeb-2 the *formatting* group—bullet-like lines, lines that break mid-sentence, spacing errors, ellipses, symbol streaks—carries most of the separation, which matches what FineWeb-2's own filters (Gopher repetition and quality rules, C4 line rules) remove. The remaining groups each contribute at most ∼0.01 AUC on this data but are what separates the hand-written junk samples (HTML, mojibake, tables, logs) that FineWeb-2's sample rarely contains.

removed group	swe	deu	fin	tur	kat	arb	hin	jpn
none (full)	0.928	0.901	0.941	0.901	0.949	0.883	0.857	0.896
noise	0.925	0.903	0.943	0.897	0.948	0.880	0.856	0.896
repetition	0.928	0.901	0.941	0.899	0.946	0.873	0.849	0.893
composition	0.925	0.899	0.942	0.900	0.948	0.883	0.857	0.894
formatting	0.847	0.839	0.905	0.870	0.923	0.809	0.824	0.844
tokens	0.924	0.902	0.944	0.901	0.949	0.889	0.857	0.896
structure	0.928	0.901	0.940	0.899	0.946	0.882	0.851	0.896
length	0.928	0.900	0.940	0.898	0.947	0.879	0.855	0.891

Table 2: Leave-one-group-out ablation, AUC kept vs removed.

4.4 A 1.5k-parameter sentiment classifier for SST-2

Task. SST-2 is the binary Stanford Sentiment Treebank [2]: single movie-review sentences, lowercased and pre-tokenised (`does n't`), labelled positive or negative. We use the GLUE split (6,919 train / 872 dev / 1,821 test). It is hard for lexicon methods because of negation ("not a bad film at all"), contrast ("boring at first but ultimately moving") and intensified negatives used as praise ("painfully funny").

Architecture. Tokens → embedding $V\times 6$ → a gated recurrent cell with state [pos_acc, neg_acc, neg_gate, int_g → Linear($4 \to 6$), ReLU, Linear($6 \to 1$), sigmoid. The six embedding axes are *pos*, *neg* (strengths 1–3), *negator*, *intensifier*, *contrast* and *hedge*; the embedding is therefore a typed sentiment lexicon of about 1,100 words. The cell is ten scalars, each a statement about English: the negation gate decays by 0.72 per token (reaching about three words) and is cancelled by a following hedge ("not *just* hilarious"); a contrast word multiplies the accumulated history by 0.35; a word under negation is routed to the opposite accumulator at 0.65 (pos→neg) or 0.75 (neg→pos) of its strength; intensifiers multiply by $1 + \text{int_gate}$, hedges by $1 - 0.5\,\text{hedge}$. All clipping is built from ReLUs. The

readout's six units are net sentiment, net negativity, any positivity, any negativity, emotional mass and dangling negation, with output weights $[1.2, -1.2, 0.15, -0.35, -0.05, -0.4]$ and bias -0.15 (the prior, stated before evaluation, that non-evaluative review sentences lean negative). $\|\theta\|_0 = 1{,}463$ blind, 1,567 after revision.

Protocol. The weights were written blind, evaluated once on the sealed test set, then revised once after reading 45 development-set errors. Trained baselines (binary unigram+bigram counts; logistic regression and a 32-unit MLP, scikit-learn defaults) were fitted on random subsets of the training set.

Weights	Labels seen	$\|\theta\|_0$	Test acc.
Hand-written	0	1,463	**78.6%**
Hand-written, one revision	872 (dev)	1,567	**79.2%**
Trained logistic regression, uni+bigram	512	9,525	64.2%
Trained MLP (32 hidden)	512	304,832	65.3%
Trained logistic regression	2,048	30,884	74.0%
Trained MLP	2,048	988,320	75.2%
Trained logistic regression	6,919	81,032	80.0%
Trained MLP	6,919	2,593,056	81.9%

Table 3: SST-2 test accuracy (1,821 sentences). The blind network is worth about 3,000 labelled sentences (Figure 2).

The revision. Five changes, each a claim about language: (i) *never*, *nothing*, *hardly* had been listed both as negators and as negative words, so "never boring" counted negative twice; (ii) the negation decay was raised from 0.55 to 0.72; (iii) the neg→pos flip from 0.45 to 0.75 after "not a bad film at all" scored 0.22; (iv) the hedge-cancels-negation term was added; (v) about 100 lexicon entries were added (comparatives, horror-positive words such as *creepiest*, review negatives such as *familiar*, *tries*, and intensifier readings of *painfully*, *brutally*). Dev accuracy went 79.6 → 81.4%, test 78.6 → 79.2%; the gap between the two gains is the honest measure of dev-set overfitting.

Where it stops. The remaining errors are largely non-lexical: sentences with no evaluative word, topic words that look like sentiment ("a compelling anatomy of *grief*" is labelled negative), irony, and polarity carried by syntax ("what the director can't do is make either persona interesting"). A lexicon-and-gates network cannot represent the last two, which is why pretrained transformers reach 95%+ on this benchmark while the designed network stops near 80%.

4.5 MNIST with weights that are drawings

Task. MNIST digits, 10,000 test images. A trained linear classifier reaches ∼92%, a small MLP ∼97%, a CNN >99%.

Architecture. Image → fixed 5×5 Gaussian blur → zero-mean, unit-norm → Conv2d($1 \to 36$, 28×28, padding 3) → max-pool over the 7×7 offsets (± 3 px translation tolerance) → maxout $36 \to 10$ (best-matching style of each digit) → class bias → argmax. Each of the 36 kernels is a 14×14 ASCII drawing typed by the author (Figure 1), expanded by a parameter-free function:

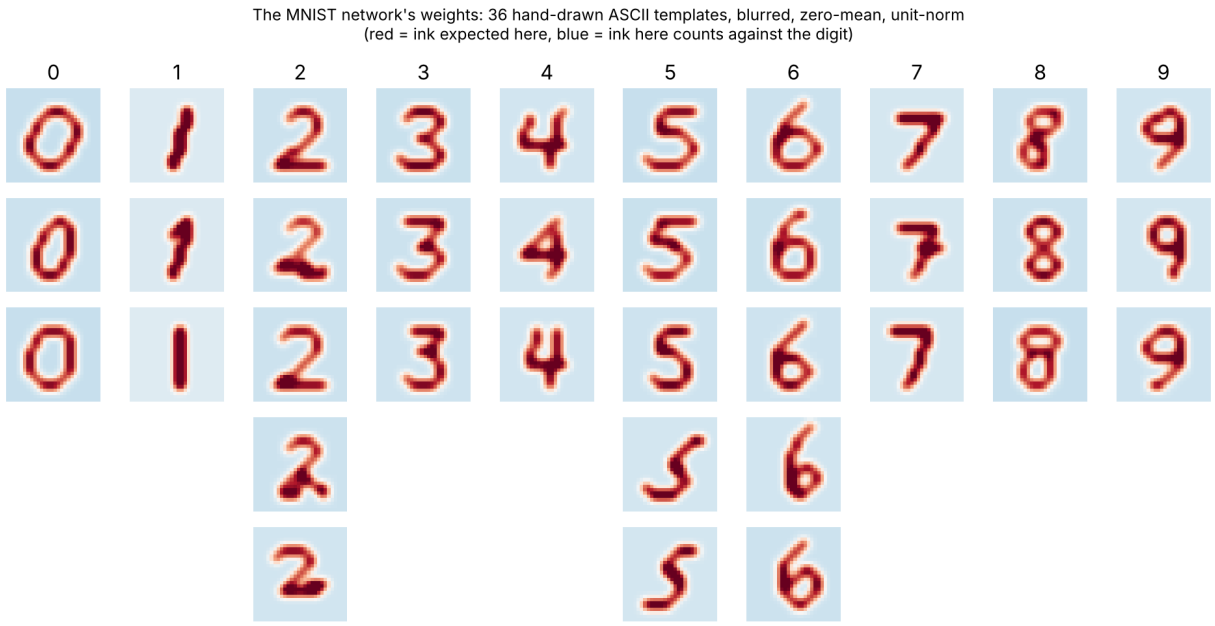


Figure 1: The MNIST network's weights: 36 hand-drawn templates after expansion to 28×28 kernels. Red: ink expected; blue: ink here is evidence against the digit.

upsample $\times 2$, widen the stroke to $\sim$3 px with a cross kernel, blur, subtract the mean (so ink *outside* the stroke counts against the match), normalise (so the activation is a cosine). Two to five styles were drawn per digit (a straight and a slanted 1, a 7 with and without a bar, an open and a closed 4, a 2 with a flat and with a looped base). We count the typed # characters as the non-zero parameters: 544 blind, 662 final.

Weights	Labels seen	Ink pixels / templates	Test acc.
Hand-drawn	0	544 / 28	**55.4%**
Hand-drawn, revision 1	500 (class means viewed)	553 / 30	**74.3%**
Hand-drawn, revision 2	500 (error images viewed)	662 / 36	**79.4%**
Trained logistic regression	50	–	61.4%
Trained logistic regression	100	–	71.5%
Trained MLP (64 hidden)	100	–	71.8%
Trained logistic regression	500	–	85.0%
Trained MLP (64 hidden)	500	–	86.2%
Trained logistic regression	10,000	–	90.9%
Trained MLP (64 hidden)	10,000	–	95.4%

Table 4: MNIST test accuracy. The best hand-drawn network is worth about 200 labelled images.

Revisions. The blind drawings were systematically wrong: one grid row too high, upright rather than right-leaning, too wide. Revision 1 redrew all ten digits after viewing the class-mean image of 500 labelled training digits next to the templates (+19 points). Revision 2 added two styles each for 2, 5 and 6 (the weakest classes) after viewing six misclassified training examples of each, widened

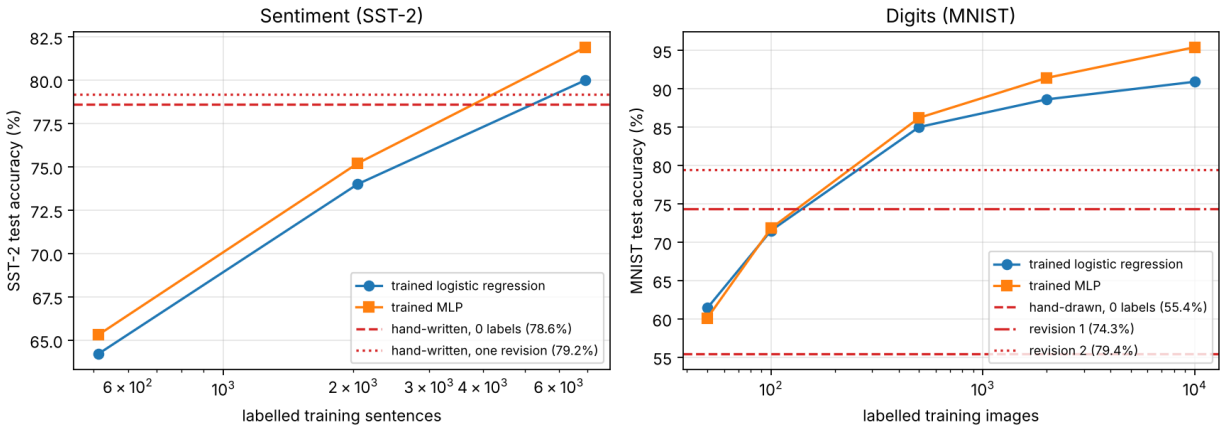


Figure 2: Designed versus trained, per label. Dashed lines: hand-written networks; curves: the same architectures trained on 2^k labels. Sentiment (left) and MNIST (right) sit at the two ends of the "nameable features" axis.

the blur from 3×3 to 5×5 and the shift from ± 2 to ± 3 px (+5 points).

Why this one loses. The large confusions are $3 \to 5$ (197), $4 \to 9$ (158), $9 \to 4$ (129) and $5 \to 8$ (106): pairs that differ in the presence of one small stroke, which a whole-digit correlation barely registers against the shared bulk of the shape. A trained linear classifier puts large weight on exactly those few discriminative pixels; a person drawing a prototype cannot know which pixels those are without seeing thousands of examples. The features that matter on MNIST are not nameable, so the design step has nothing to work with. A part-based variant (hand-drawn stroke detectors feeding a hand-written 10×64 linear layer) was tried and abandoned: writing those 640 numbers was, on inspection, guessing.

5 Discussion and limits

5.1 When does writing beat training?

The three networks line up on one axis: how *nameable* the discriminative features are (Table 5). Where an expert could write the rules as a list —this word is positive, negation reaches three words, mojibake looks like Ã¶—the rules map one-to-one onto units and the designed network is competitive with thousands of labels. Where the features are statistical regularities in pixel space that nobody can enumerate, a few hundred labels beat the best drawings anyone can make. Lauritzen's legal-citation task [1] sits at the far nameable end (reporter names, volume and page formats), which is why designed weights beat 8,192 labels there.

5.2 The revision step is where the labels go

A revision after inspecting 500 labelled examples is not zero-label learning; it is hypothesis-driven learning with very few degrees of freedom. The right comparison for a revised designed network is a trained one at the same label count, which is why every table lists labels seen. On MNIST the revised network (500 seen, 79.4%) loses to a trained one on 500 labels (85–86%); on SST-2 the revised network (872 seen, 79.2%) beats a trained one on 2,048.

	Sentiment	Text quality	Digits
Input unit	word	character attribute	pixel
Nameable features?	yes: lexicon, negation, contrast	yes: markup, repeats, mojibake, lengths	no
Designed vs trained	wins up to ∼3,000 labels	AUC 0.86–0.95, no labels	loses beyond ∼200 labels
Ceiling	syntax, irony	grammar	fine stroke differences

Table 5: Where hand-written weights are competitive.

5.3 A capability measurement

"Can model M write the weights of a working classifier for task T?" has a numeric answer; it differs across tasks in an interpretable way; and it will presumably rise across model generations. A model that writes a 79% MNIST classifier from an ASCII-art prior and a 79% SST-2 classifier from its own lexicon is telling us something about what it knows that a multiple-choice benchmark does not.

5.4 Things that went wrong

Dual-role words (*never* as negator and as negative) need an explicit decision. The blind MNIST drawings were one row too high; a single *unlabelled* look at the input distribution would have prevented this and a future protocol should allow it. "Painfully funny" was scored as *painful*+*funny*; intensifier readings of negative adverbs are real in review English. A document-length penalty appropriate for pretraining filtering dominates single sentences. And when a weight cannot be justified in a sentence—as in the abandoned part-based MNIST net—that is the signal to stop.

6 Conclusion

References

[1] Jacob Lauritzen. Got fable to hand-write weights of a 100k param model to recognize real versus fake legal citations. X (Twitter) post, @jacsebl, November 2026.

[2] Richard Socher, Alex Perelygin, Jean Wu, Jason Chuang, Christopher D. Manning, Andrew Ng, and Christopher Potts. Recursive deep models for semantic compositionality over a sentiment treebank. In *EMNLP*, 2013.